PYTHON ACTIVITY BASED LEARNING

1. Problem Statement

A college wants to select students for a scholarship program when only a fixed number of scholarships are available. The selection process must be completed in multiple stages.

First, students must satisfy the minimum academic marks requirement, attendance requirement, and family income limit. Students who satisfy all three conditions become eligible. The eligible students are then ranked according to their academic marks, and only the top-ranked students are selected according to the available scholarship quota.

The Python program automates this shortlisting process and displays the selected students and their ranks.

2. Objectives

- To check the minimum academic eligibility of students.
- To verify attendance requirements.
- To check whether family income is within the permitted limit.
- To rank eligible students according to their marks.
- To select candidates according to a fixed scholarship quota.
- To display selected and non-selected students clearly.

3. Assumptions

The following rules are assumed for this case study:

Requirement	Condition
Minimum academic marks	75 out of 100
Minimum attendance	75%
Maximum annual family income	₹2,50,000

Scholarship quota	3 students
Ranking rule	Higher marks receive a higher rank

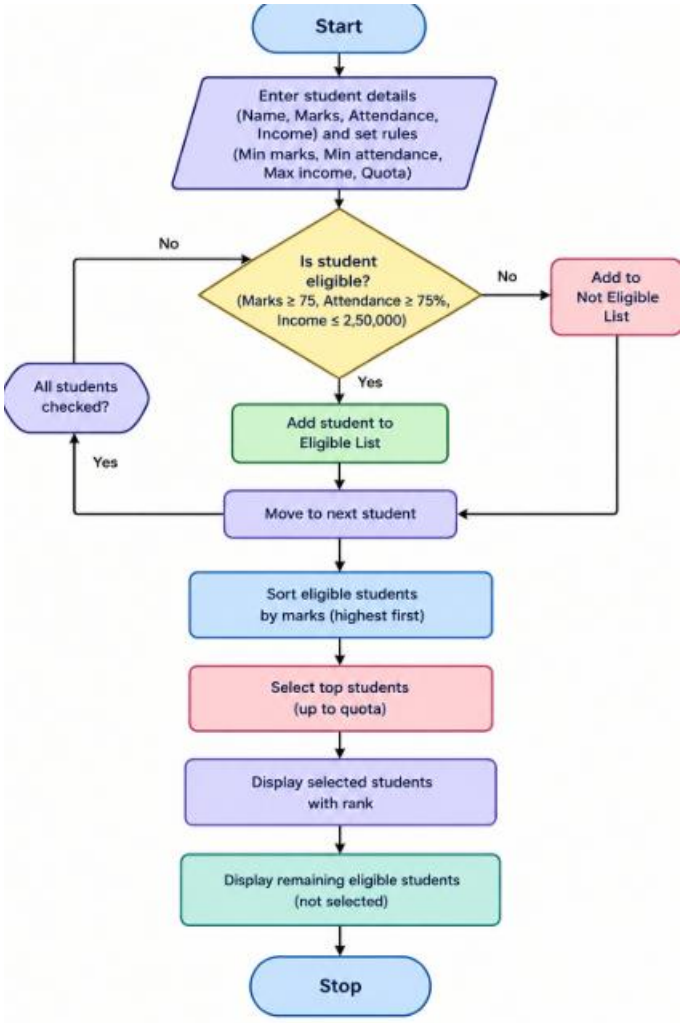
- Students must satisfy all three eligibility conditions.
- If eligible students exceed the quota, only the top three are selected.
- If two students have equal marks, their input order determines their ranking.
- The sample student information is used for demonstration.

4. Algorithm

1. Start the program.
2. Store the student details, including name, marks, attendance, and annual family income.
3. Set the minimum marks to 75, minimum attendance to 75%, maximum family income to ₹2,50,000, and scholarship quota to 3.
4. Check each student's marks, attendance, and family income.
5. Add students who satisfy all conditions to the eligible list.
6. Sort the eligible students in descending order of marks.
7. Select the top three eligible students according to the quota.
8. Display the selected students, their marks, and their ranks.
9. Display the remaining eligible students as not selected due to the quota.
10. Stop the program.

5. Flowchart

https://chatgpt.com/s/m_6ae06f299c2481919a4cc53d47b1ec4d



6. pseudocode

START

SET minimum_marks = 75

SET minimum_attendance = 75

SET maximum_income = 250000

SET quota = 3

CREATE an empty eligible list

FOR each student:

IF marks >= minimum_marks AND

attendance >= minimum_attendance AND

income <= maximum_income:

ADD student to eligible list

SORT eligible students by marks in descending order

DISPLAY "Scholarship Selected Students"

FOR the first quota students:

DISPLAY student name, marks and rank

DISPLAY remaining eligible students as not selected

STOP

7. Python Code

Copy this complete program into Python IDLE, VS Code, or another Python editor.

```
# Multi-Stage Scholarship Shortlisting

students = [
    ("Ravi", 85, 90, 200000),
```

```
    ("Sita", 92, 95, 180000),
    ("Kiran", 78, 80, 220000),
    ("Priya", 88, 70, 150000),
    ("Arun", 95, 90, 300000),
    ("Meena", 90, 85, 200000),
    ("Raju", 80, 78, 240000)
]

minimum_marks = 75
minimum_attendance = 75
maximum_income = 250000
quota = 3

eligible = []

# Stage 1: Check eligibility
for student in students:
    name, marks, attendance, income = student

    if (marks >= minimum_marks and
        attendance >= minimum_attendance and
        income <= maximum_income):
        eligible.append(student)

# Stage 2: Rank eligible students
eligible.sort(key=lambda student: student[1], reverse=True)
```

```
# Stage 3: Select students within the quota
selected = eligible[:quota]

print("SCHOLARSHIP SHORTLIST")
print("---------------------")

for rank, student in enumerate(selected, start=1):
    name, marks, attendance, income = student
    print("Rank:", rank)
    print("Name:", name)
    print("Marks:", marks)
    print("Attendance:", str(attendance) + "%")
    print("Family Income: Rs.", income)
    print()

print("Not Selected:")
for student in eligible[quota:]:
    print(student[0])

print("\nTotal Eligible:", len(eligible))
print("Total Selected:", len(selected))
```

8. Test Cases

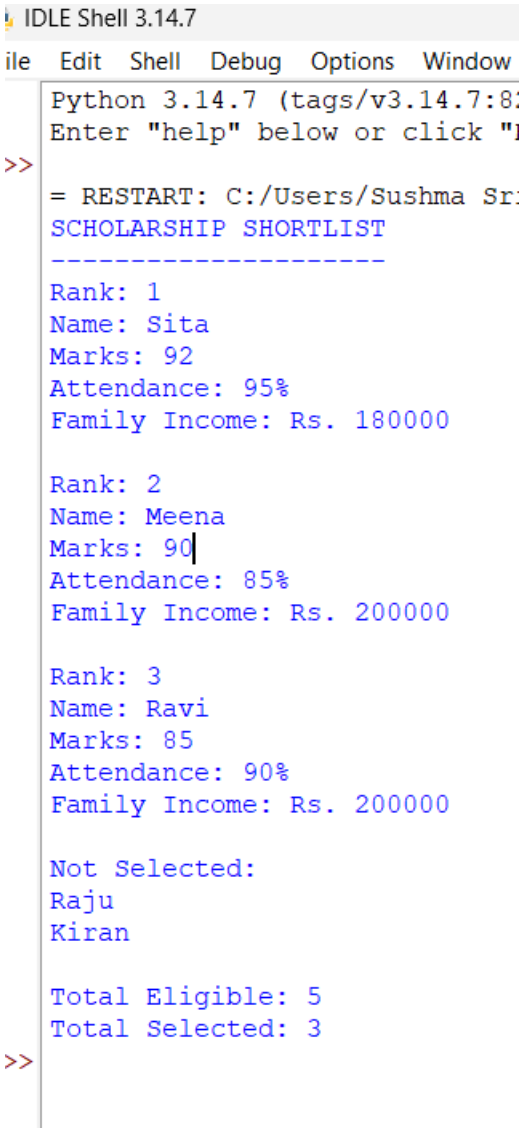
Test case Marks Attendance Annual income Expected result

1	85	90%	₹2,00,000	Eligible
2	92	95%	₹1,80,000	Eligible
3	78	80%	₹2,20,000	Eligible
4	88	70%	₹1,50,000	Not eligible: attendance
5	95	90%	₹3,00,000	Not eligible: income
6	90	85%	₹2,00,000	Eligible
7	80	78%	₹2,40,000	Eligible

These cases test academic eligibility, attendance, income, ranking, and the scholarship quota.

9. Output Screenshots

The following is the expected console output for the program. You can run the code and take an actual screenshot for the final PDF.



10. Conclusion

The Multi-Stage Scholarship Shortlisting program successfully identifies eligible students using academic marks, attendance, and annual family income. It ranks the eligible students based on their marks and selects only the top three students according to the fixed scholarship quota. This program demonstrates the practical use of conditional statements, lists, loops, sorting, and ranking in Python.

Case study-2

1. Problem Statement

A university needs to verify whether students have fulfilled the requirements to attend their examinations. The university must check fee payments, attendance percentage, internal marks, and examination eligibility records.

The program identifies students who have outstanding conditions, such as unpaid fees, low attendance, or insufficient internal marks. It generates a reason-wise report showing which conditions each student has not fulfilled.

2. Objectives

- To check whether students have paid their fees.
- To verify minimum attendance requirements.
- To check minimum internal marks.
- To identify students who meet all examination requirements.
- To list outstanding conditions for each student.
- To generate a reason-wise report for pending requirements.

3. Assumptions

Requirement	Rule
Fee payment	Full fees must be paid
Minimum attendance	75%
Minimum internal marks	40 out of 100
Examination eligibility	All three conditions must be satisfied
Report	Display each student's pending

These are sample rules for the assignment. Actual university policies may differ.

5. Flowchart

Use this flowchart image in your PDF under Section 5.

Save the flowchart image

Insert the saved image into Word or Google Docs, then export your document as PDF.

6. Pseudocode

START

SET minimum attendance = 75

SET minimum internal marks = 40

CREATE eligible list

CREATE outstanding list

CREATE reason counts

FOR each student:

  CREATE empty reasons list

  IF fees are not paid:

    ADD "Fee not paid" to reasons

  IF attendance < 75:

    ADD "Low attendance" to reasons

  IF internal marks < 40:

    ADD "Low internal marks" to reasons

  IF exam status is not eligible:

    ADD "Not eligible for examination" to reasons

IF reasons list is empty:

ADD student to eligible list

ELSE:

ADD student and reasons to outstanding list

COUNT each reason

DISPLAY eligible students

DISPLAY outstanding students with reasons

DISPLAY reason-wise counts

STOP

7. Python Code

Copy the complete code into a file named university_audit.py in VS Code or Python IDLE.

```
# Integrated University Fee and Eligibility Audit

students = [
  {"name": "Ravi", "fees": "Paid",
   "attendance": 85, "marks": 65, "exam": "Yes"},

  {"name": "Sita", "fees": "Pending",
   "attendance": 90, "marks": 70, "exam": "No"},
```

```
    {"name": "Kiran", "fees": "Paid",
     "attendance": 68, "marks": 55, "exam": "Yes"},

    {"name": "Meena", "fees": "Paid",
     "attendance": 85, "marks": 35, "exam": "Yes"},

    {"name": "Arun", "fees": "Pending",
     "attendance": 60, "marks": 30, "exam": "No"}
]

eligible = []
outstanding = []
reason_count = {}

# Check every student
for student in students:
    reasons = []

    if student["fees"] != "Paid":
        reasons.append("Fee not paid")

    if student["attendance"] < 75:
        reasons.append("Low attendance")

    if student["marks"] < 40:
        reasons.append("Low internal marks")
```

```
    if student["exam"] != "Yes":
        reasons.append("Not eligible for examination")

    if len(reasons) == 0:
        eligible.append(student["name"])
    else:
        outstanding.append((student["name"], reasons))

        for reason in reasons:
            reason_count[reason] = reason_count.get(reason, 0) + 1

# Display the audit report
print("UNIVERSITY FEE AND ELIGIBILITY AUDIT")
print("------------------------------------")

print("\nEligible Students:")
if eligible:
    for name in eligible:
        print(name)
else:
    print("No eligible students")

print("\nStudents with Outstanding Conditions:")
for name, reasons in outstanding:
    print(name, ":", ", ".join(reasons))
```

```
print("\nReason-wise Report:")
for reason, count in reason_count.items():
   print(reason, ":", count, "student(s)")


print("\nTotal Students:", len(students))
print("Eligible Students:", len(eligible))
print("Students with Outstanding Conditions:",
     len(outstanding))
```

8. Test Cases

Student	Fees	Attendance	Internal marks	Expected result
Ravi	Paid	85%	65	Eligible
Sita	Pending	90%	70	Fee and exam conditions outstanding
Kiran	Paid	68%	55	Low attendance
Meena	Paid	85%	35	Low internal marks
Arun	Pending	60%	30	Multiple outstanding conditions

9. Output Screenshot

The following is the expected output of the program. Run the code in VS Code or IDLE to capture your actual output screenshot for the assignment.

```
========== RESTART: C:/Users/Sushma Sri/AppData/I
UNIVERSITY FEE AND ELIGIBILITY AUDIT
------------------------------------

Eligible Students:
Ravi

Students with Outstanding Conditions:
Sita : Fee not paid, Not eligible for examination
Kiran : Low attendance
Meena : Low internal marks
Arun : Fee not paid, Low attendance, Low internal marks, Not eligible for examination

Reason-wise Report:
Fee not paid : 2 student(s)
Not eligible for examination : 2 student(s)
Low attendance : 2 student(s)
Low internal marks : 2 student(s)

Total Students: 5
Eligible Students: 1
Students with Outstanding Conditions: 4
>>>
```

10. Conclusion

The Integrated University Fee and Eligibility Audit program checks student fee payments, attendance, internal marks, and examination eligibility. It identifies eligible students and students with outstanding conditions. The reason-wise report helps the university understand which requirements need attention. This program demonstrates the practical use of Python lists, dictionaries, loops, conditional statements, and counting techniques.

Case study 3:

1. Problem Statement

A university hostel needs to monitor the electricity and water consumption of its rooms. The hostel management wants to calculate the charges for each room, identify rooms that exceed the permitted consumption limits, and compare the total resource usage of different floors.

The program analyses electricity consumption in kilowatt-hours (kWh) and water consumption in kilolitres (kL). It calculates the charges, reports threshold violations, and displays floor-wise resource utilization to help management reduce wastage

2. Objectives

- To record electricity and water consumption for each hostel room.
- To calculate room-wise electricity and water charges.
- To identify rooms that exceed consumption thresholds.
- To calculate total resource usage for each floor.
- To compare electricity and water consumption across floors.
- To generate a clear resource monitoring report.

3. Assumptions

Parameter	Assumed value
Electricity charge	₹8 per kWh
Water charge	₹25 per kL
Electricity threshold	150 kWh per room
Water threshold	10 kL per room
Consumption period	One month

- Electricity and water charges are calculated separately.
- A room violates a threshold only when its consumption is greater than the limit.
- Floor totals include every room assigned to that floor.
- Sample room readings are used for demonstration.

4. Algorithm

1. Start the program.
2. Store the room number, floor number, electricity usage, and water usage.
3. Set the electricity and water rates and their consumption limits.
4. Calculate electricity charges and water charges for each room.
5. Calculate the total charge for each room.
6. Check whether electricity consumption exceeds 150 kWh.
7. Check whether water consumption exceeds 10 kL.

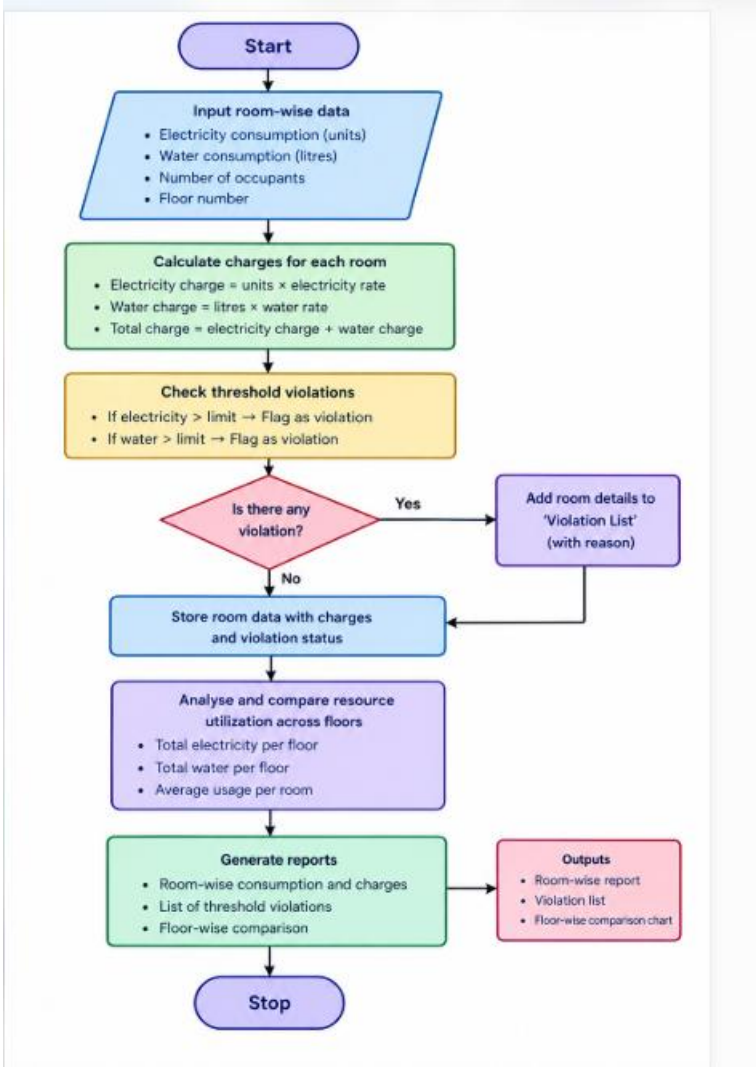
8. Record any threshold violations and their reasons.
9. Calculate total electricity usage, water usage, and charges for each floor.
10. Display the room-wise report, violation report, and floor-wise comparison.
11. Stop the program.

5. Flowchart

Smart Hostel Resource Monitoring — Flowchart

Save the image below and insert it under Section 5 in your PDF.

Open or save the flowchart image



6. Pseudocode

START

SET electricity_rate = 8

SET water_rate = 25

SET electricity_limit = 150

SET water_limit = 10

FOR each hostel room:

  CALCULATE electricity charge

  CALCULATE water charge

  CALCULATE total charge

  CREATE empty violation list

  IF electricity > electricity_limit:

    ADD electricity violation

  IF water > water_limit:

    ADD water violation

  DISPLAY room details and charges

  DISPLAY violation reasons if any

  ADD room usage to its floor totals

DISPLAY floor-wise electricity consumption

DISPLAY floor-wise water consumption

DISPLAY floor-wise total charges

STOP

7. Python Code

Copy this entire program into VS Code or Python IDLE and save it as hostel_monitoring.py.

```
# Smart Hostel Resource Monitoring

rooms = [
  {"room": "A101", "floor": 1, "electricity": 120, "water": 8},
  {"room": "A102", "floor": 1, "electricity": 180, "water": 12},
  {"room": "B201", "floor": 2, "electricity": 140, "water": 9},
  {"room": "B202", "floor": 2, "electricity": 200, "water": 7},
  {"room": "C301", "floor": 3, "electricity": 100, "water": 11},
  {"room": "C302", "floor": 3, "electricity": 160, "water": 10}
]

electricity_rate = 8
water_rate = 25
electricity_limit = 150
water_limit = 10

floor_data = {}
total_charges = 0
```

```
print("SMART HOSTEL RESOURCE MONITORING")
print("--------------------------------")

# Analyse each room
for room in rooms:
    electricity = room["electricity"]
    water = room["water"]

    electricity_charge = electricity * electricity_rate
    water_charge = water * water_rate
    charge = electricity_charge + water_charge
    total_charges += charge

    violations = []

    if electricity > electricity_limit:
        violations.append("High electricity usage")

    if water > water_limit:
        violations.append("High water usage")

    print("\nRoom:", room["room"])
    print("Floor:", room["floor"])
    print("Electricity:", electricity, "kWh")
    print("Water:", water, "kL")
```

```
    print("Electricity Charge: Rs.", electricity_charge)
    print("Water Charge: Rs.", water_charge)
    print("Total Charge: Rs.", charge)

    if violations:
        print("Violation:", ", ".join(violations))
    else:
        print("Violation: None")

    # Store floor-wise totals
    floor = room["floor"]

    if floor not in floor_data:
        floor_data[floor] = {
            "electricity": 0,
            "water": 0,
            "charges": 0
        }

    floor_data[floor]["electricity"] += electricity
    floor_data[floor]["water"] += water
    floor_data[floor]["charges"] += charge

# Display floor-wise comparison
print("\nFLOOR-WISE RESOURCE COMPARISON")
print("--------------------------------")
```

```
for floor, data in sorted(floor_data.items()):
    print("\nFloor:", floor)
    print("Electricity:", data["electricity"], "kWh")
    print("Water:", data["water"], "kL")
    print("Total Charges: Rs.", data["charges"])

print("\nTotal Hostel Charges: Rs.", total_charges)
```

8. Test Cases

Room	Electricity	Water	Expected result
A101	120 kWh	8 kL	No violation
A102	180 kWh	12 kL	Both limits
B201	140 kWh	9 kL	No violation
B202	200 kWh	7 kL	Electricity limit exceeded
C301	100 kWh	11 kL	Water limit exceeded
C302	160 kWh	10 kL	Electricity limit exceeded

9. Output Screenshot

This is a sample of the expected output from the Python program. Run it in VS Code or IDLE to capture your actual output screenshot.

```
ile  Edit  Shell  Debug  Options  Window  Help
>>
========== RESTART: C:/Users/Sushma Sri/AppData/Local/Programs/Pyt
SMART HOSTEL RESOURCE MONITORING
--------------------------------

Room: A101
Floor: 1
Electricity: 120 kWh
Water: 8 kL
Electricity Charge: Rs. 960
Water Charge: Rs. 200
Total Charge: Rs. 1160
Violation: None

Room: A102
Floor: 1
Electricity: 180 kWh
Water: 12 kL
Electricity Charge: Rs. 1440
Water Charge: Rs. 300
Total Charge: Rs. 1740
Violation: High electricity usage, High water usage

Room: B201
Floor: 2
Electricity: 140 kWh
Water: 9 kL
Electricity Charge: Rs. 1120
Water Charge: Rs. 225
Total Charge: Rs. 1345
Violation: None

Room: B202
Floor: 2
Electricity: 200 kWh
Water: 7 kL
Electricity Charge: Rs. 1600
Water Charge: Rs. 175
Total Charge: Rs. 1775
Violation: High electricity usage

Room: C301
Floor: 3
Electricity: 100 kWh
Water: 11 kL
Electricity Charge: Rs. 800
Water Charge: Rs. 275
```

```
Total Charge: Rs. 1530
Violation: High electricity usage

FLOOR-WISE RESOURCE COMPARISON
--------------------------------

Floor: 1
Electricity: 300 kWh
Water: 20 kL
Total Charges: Rs. 2900

Floor: 2
Electricity: 340 kWh
Water: 16 kL
Total Charges: Rs. 3120

Floor: 3
Electricity: 260 kWh
Water: 21 kL
Total Charges: Rs. 2605

Total Hostel Charges: Rs. 8625
```

10. Conclusion

The Smart Hostel Resource Monitoring program calculates electricity and water charges for individual hostel rooms. It identifies rooms that exceed consumption limits and compares resource usage across floors. This program helps hostel management monitor resource utilization, identify possible wastage, and understand room-wise and floor-wise expenses. It demonstrates the use of Python dictionaries, lists, loops, conditional statements, and arithmetic operations.

case study 4:

1. Problem Statement

A courier company receives packages that must be delivered to different destination zones. Each package has a specific weight and delivery priority, while each delivery vehicle has a fixed carrying capacity and a designated destination zone.

The company needs a Python program to allocate packages to suitable vehicles without exceeding their capacities. High-priority packages should be considered first. Packages

that cannot be assigned to a suitable vehicle must be listed in a separate report with the reason for non-dispatch.

2. Objectives

- To store package weights, destination zones, and delivery priorities.
- To maintain vehicle capacities and destination zones.
- To allocate high-priority packages first.
- To ensure that vehicle capacity limits are not exceeded.
- To identify packages that cannot be dispatched.
- To display vehicle-wise allocations and a non-dispatch report.

3. Assumptions

Parameter	Assumption
Package weight	Measured in kilograms
Vehicle capacity	Maximum carrying weight in kilograms
Destination zones	North, South, and East
Delivery priority	High, Medium, or Low
Allocation rule	Package zone must match vehicle zone
Priority rule	High before Medium before Low
Capacity rule	Total allocated weight must not exceed capacity

Each vehicle serves one destination zone. A package that cannot fit into any available vehicle for its zone remains undispatched. Packages with equal priority are processed in their original input order.

4. Algorithm

1. Start the program.
2. Store package details and vehicle details.
3. Arrange packages in priority order: High, Medium, and Low.

4. Check whether a vehicle is available for the package's destination zone.
5. Check whether the package fits within the vehicle's remaining capacity.
6. If both conditions are satisfied, allocate the package and update the vehicle's load.
7. Otherwise, record the package as undispatched and mention the reason.
8. Repeat the process for all packages.
9. Display vehicle-wise package allocations and total weights.
10. Display the undispatched package list with reasons.
11. Stop the program.

5. Flowchart

Multi-Route Courier Dispatch Planning — Flowchart

Open or save the flowchart image

Save the image and insert it under Section 5 of your assignment PDF.

6. Pseudocode

START

START

CREATE package list

CREATE vehicle list

SET priority order = High, Medium, Low

SORT packages by priority

FOR each package:

FIND vehicles matching the destination zone

IF no matching vehicle exists:

RECORD "No vehicle for destination zone"

ELSE:

FIND a matching vehicle with enough capacity

IF vehicle is found:

ALLOCATE package to vehicle

UPDATE vehicle load

ELSE:

RECORD "Insufficient remaining capacity"

DISPLAY vehicle-wise allocations

DISPLAY total weight of each vehicle

DISPLAY undispatched packages with reasons

STOP

7. Python Code

Copy this complete program into VS Code or Python IDLE and save it as courier_dispatch.py.

```
# Multi-Route Courier Dispatch Planning

packages = [
```

```
    {"id": "P101", "weight": 6, "zone": "North",
     "priority": "High"},
    {"id": "P105", "weight": 9, "zone": "North",
     "priority": "High"},
    {"id": "P103", "weight": 4, "zone": "South",
     "priority": "High"},
    {"id": "P106", "weight": 2, "zone": "West",
     "priority": "Medium"},
    {"id": "P102", "weight": 3, "zone": "North",
     "priority": "Medium"},
    {"id": "P104", "weight": 5, "zone": "East",
     "priority": "Low"},
    {"id": "P107", "weight": 5, "zone": "South",
     "priority": "Low"}
]

vehicles = [
    {"id": "V1", "zone": "North", "capacity": 10,
     "load": 0, "packages": []},
    {"id": "V2", "zone": "South", "capacity": 8,
     "load": 0, "packages": []},
    {"id": "V3", "zone": "East", "capacity": 7,
     "load": 0, "packages": []}
]

priority_order = {"High": 1, "Medium": 2, "Low": 3}
```

```
# Process high-priority packages first
packages.sort(
   key=lambda package: priority_order[package["priority"]]
)

undispatched = []

for package in packages:
   matching_vehicles = []

   for vehicle in vehicles:
      if vehicle["zone"] == package["zone"]:
         matching_vehicles.append(vehicle)

   if len(matching_vehicles) == 0:
      reason = "No vehicle for destination zone"
      undispatched.append((package, reason))
      continue

   allocated = False

   for vehicle in matching_vehicles:
      if (vehicle["load"] + package["weight"]
            <= vehicle["capacity"]):
         vehicle["packages"].append(package["id"])
```

```
            vehicle["load"] += package["weight"]
            allocated = True
            break

    if not allocated:
        reason = "Insufficient remaining capacity"
        undispatched.append((package, reason))

# Display vehicle-wise report
print("COURIER DISPATCH REPORT")
print("----------------------")

for vehicle in vehicles:
    print("\nVehicle:", vehicle["id"])
    print("Destination:", vehicle["zone"])
    print("Packages:", vehicle["packages"])
    print("Total Weight:", vehicle["load"], "kg")
    print("Capacity:", vehicle["capacity"], "kg")

# Display undispatched packages
print("\nUNDISPATCHED PACKAGES")
print("---------------------")

if len(undispatched) == 0:
    print("All packages dispatched")
else:
```

```
    for package, reason in undispatched:
        print("Package:", package["id"])
        print("Zone:", package["zone"])
        print("Weight:", package["weight"], "kg")
        print("Priority:", package["priority"])
        print("Reason:", reason)
        print()

print("Total Packages:", len(packages))
print("Dispatched:", len(packages) - len(undispatched))
print("Undispatched:", len(undispatched))
```

8. Test Cases

Package	Weight	Zone	Priority	Expected result
P101	6 kg	North	High	Dispatched to V1
P105	9 kg	North	High	Not dispatched: insufficient capacity
P103	4 kg	South	High	Dispatched to V2
P106	2 kg	West	Medium	Not dispatched: no vehicle
P102	3 kg	North	Medium	Dispatched to V1
P104	5 kg	East	Low	Dispatched to V3
P107	5 kg	South	Low	Not dispatched: insufficient capacity

These test cases check priority handling, zone matching, vehicle capacity, and undispatched package reporting.

9. Output Screenshot

The following is the expected output. Run the program in VS Code or Python IDLE and take a real screenshot for your PDF.

```
>>>
========== RESTART: C:/Use
COURIER DISPATCH REPORT
-----------------------

Vehicle: V1
Destination: North
Packages: ['P101', 'P102']
Total Weight: 9 kg
Capacity: 10 kg

Vehicle: V2
Destination: South
Packages: ['P103']
Total Weight: 4 kg
Capacity: 8 kg

Vehicle: V3
Destination: East
Packages: ['P104']
Total Weight: 5 kg
Capacity: 7 kg

UNDISPATCHED PACKAGES
---------------------
Package: P105
Zone: North
Weight: 9 kg
Priority: High
Reason: Insufficient remaining capacity

Package: P106
Zone: West
Weight: 2 kg
Priority: Medium
Reason: No vehicle for destination zone

Package: P107
Zone: South
Weight: 5 kg
Priority: Low
Reason: Insufficient remaining capacity

Total Packages: 7
Dispatched: 4
Undispatched: 3
>>>
```

10. Conclusion

The Multi-Route Courier Dispatch Planning program allocates packages to vehicles based on destination zones, delivery priorities, and vehicle capacities. It processes high-priority packages first and ensures that vehicle capacity limits are not exceeded.

Packages that cannot be dispatched are reported with clear reasons. This program demonstrates the practical use of Python lists, dictionaries, sorting, loops, and conditional statements in courier logistics.

Case study5:

1. Problem Statement

A college organizes a campus festival in which students register for different activities. The organizing committee collects registration fees and sponsorship contributions and spends money on decorations, food, prizes, and other festival arrangements.

The committee needs a Python program to reconcile participant records, identify duplicate registrations, calculate the total registration fees collected, add sponsorship contributions, calculate total expenses, and determine the final financial balance.

The program should avoid counting duplicate registration records more than once and generate a clear financial audit report.

2. Objectives

- To maintain participant registration records.
- To identify duplicate registration IDs.
- To calculate the total registration fees collected.
- To calculate total sponsorship contributions.
- To calculate total festival expenses.
- To determine the final financial balance.
- To generate a report showing duplicate registrations and financial details.

3. Assumptions

Parameter	Assumption
Registration fee	₹500 per participant
Registration identification	Each participant has a unique registration ID

Duplicate handling	Duplicate IDs are reported and counted only once
Unpaid registration	₹0 collected until payment is received
Sponsorship	Contributions are recorded separately
Financial balance	Total income − total expenses

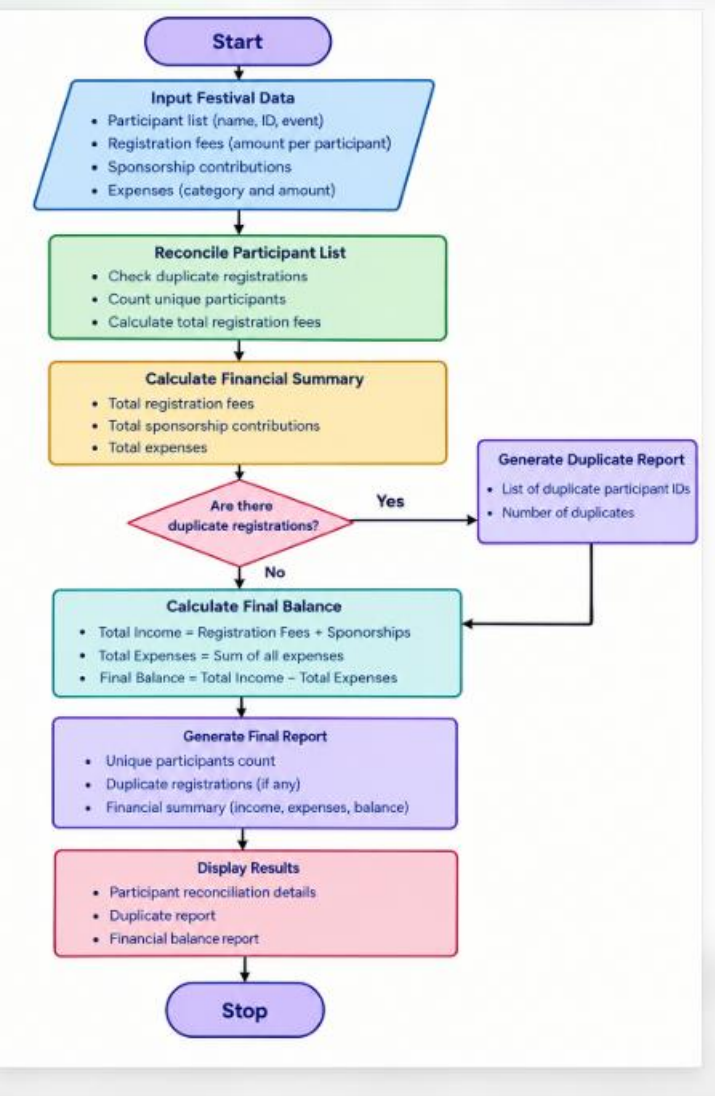
The sample data is used for demonstration. The final balance is calculated from money actually collected, not from unpaid registration fees.

4. Algorithm

1. Start the program.
2. Store participant registration records, sponsorship contributions, and festival expenses.
3. Create a set to track registration IDs already processed.
4. Check each registration ID for duplicates.
5. Count each unique registration only once and add only collected fees to the total.
6. Record duplicate registration IDs separately.
7. Add all sponsorship contributions to calculate total sponsorship income.
8. Add all expenses to calculate total expenditure.
9. Calculate total income as registration fees plus sponsorship contributions.
10. Calculate the final balance by subtracting expenses from total income.
11. Display participant details, duplicate records, income, expenses, and final balance.
12. Stop the program.

5. Flowchart

Campus Festival Registration and Financial Audit — Flowchart



6. Pseudocode

START

CREATE participant list

CREATE empty set of registered IDs

CREATE empty duplicate list

SET registration fees = 0

FOR each registration:

IF registration ID is already present:

ADD ID to duplicate list

ELSE:

ADD ID to registered IDs

ADD collected fee to registration fees

CALCULATE total sponsorship contributions

CALCULATE total expenses

total income = registration fees + sponsorships

final balance = total income - total expenses

DISPLAY unique participant count

DISPLAY duplicate registrations

DISPLAY registration fees

DISPLAY sponsorship contributions

DISPLAY total expenses

DISPLAY final balance

STOP

7. Python Code

Copy this complete program into VS Code or Python IDLE and save it as festival_audit.py

```
# Campus Festival Registration and Financial Audit

registrations = [
```

```
    ("R001", "Anu", 500),
    ("R002", "Ravi", 500),
    ("R003", "Sita", 500),
    ("R002", "Ravi", 500),  # Duplicate registration
    ("R004", "Kiran", 0),   # Fee not paid
    ("R005", "Meena", 500)
]

sponsorships = [10000, 5000]

expenses = [
    ("Decorations", 6000),
    ("Food", 3500),
    ("Prizes", 2500)
]

registered_ids = set()
duplicate_ids = []
unique_participants = []
registration_fees = 0

# Check participant registrations
for registration in registrations:
    reg_id, name, fee = registration

    if reg_id in registered_ids:
```

```
        if reg_id not in duplicate_ids:
            duplicate_ids.append(reg_id)
    else:
        registered_ids.add(reg_id)
        unique_participants.append(registration)
        registration_fees += fee

# Calculate sponsorship and expenses
total_sponsorship = sum(sponsorships)
total_expenses = sum(amount for item, amount in expenses)

total_income = registration_fees + total_sponsorship
final_balance = total_income - total_expenses

# Display participant report
print("CAMPUS FESTIVAL AUDIT")
print("---------------------")

print("\nUnique Participants:")
for reg_id, name, fee in unique_participants:
    print(reg_id, "-", name, "- Fee Paid: Rs.", fee)

print("\nTotal Unique Participants:", len(unique_participants))
print("Duplicate Registration IDs:", duplicate_ids)
print("Number of Duplicate IDs:", len(duplicate_ids))
```

```
# Display financial report
print("\nFINANCIAL REPORT")
print("----------------")
print("Registration Fees: Rs.", registration_fees)
print("Sponsorship Contributions: Rs.", total_sponsorship)

print("\nFestival Expenses:")
for item, amount in expenses:
    print(item, ": Rs.", amount)

print("Total Expenses: Rs.", total_expenses)
print("Total Income: Rs.", total_income)
print("Final Financial Balance: Rs.", final_balance)

# Display unpaid registration fees
print("\nUnpaid Registrations:")
unpaid_found = False

for reg_id, name, fee in unique_participants:
    if fee == 0:
        print(reg_id, "-", name)
        unpaid_found = True

if not unpaid_found:
    print("None")
```

8. Test Cases

Test case	Input or condition	Expected result
1	R001 registered with ₹500	Fee included
2	R002 appears twice	Duplicate detected and fee counted once
3	R004 has fee ₹0	Unpaid registration identified
4	Sponsorships of ₹10,000 and ₹5,000	Total ₹15,000
5	Expenses of ₹6,000, ₹3,500 and ₹2,500	Total ₹12,000
6	Income ₹17,000 and expenses ₹12,000	Final balance ₹5,000

9. Output Screenshot

The following is the expected output. Run the program in VS Code or Python IDLE to capture the actual screenshot for your PDF.

```
>>
   ================================================== ]
   CAMPUS FESTIVAL AUDIT
   ---------------------

   Unique Participants:
   R001 - Anu - Fee Paid: Rs. 500
   R002 - Ravi - Fee Paid: Rs. 500
   R003 - Sita - Fee Paid: Rs. 500
   R004 - Kiran - Fee Paid: Rs. 0
   R005 - Meena - Fee Paid: Rs. 500

   Total Unique Participants: 5
   Duplicate Registration IDs: ['R002']
   Number of Duplicate IDs: 1

   FINANCIAL REPORT
   ----------------
   Registration Fees: Rs. 2000
   Sponsorship Contributions: Rs. 15000

   Festival Expenses:
   Decorations : Rs. 6000
   Food : Rs. 3500
   Prizes : Rs. 2500
   Total Expenses: Rs. 12000
   Total Income: Rs. 17000
   Final Financial Balance: Rs. 5000

   Unpaid Registrations:
   R004 - Kiran
>>
```

10. Conclusion

The Campus Festival Registration and Financial Audit program reconciles participant registrations, detects duplicate registration IDs, and calculates registration fee income, sponsorship contributions, and festival expenses. It identifies unpaid registrations and calculates the final financial balance. This program demonstrates the use of Python lists, sets, loops, conditional statements, and built-in functions such as sum().

Case study:

1. Problem Statement

A college maintains student information, including student ID, name, marks, attendance percentage, and course details. Sometimes, the records may contain duplicate student IDs, missing names, invalid marks, incorrect attendance values, or missing course information.

The college needs a Python program to check the consistency of these records, identify errors, and generate a report showing valid records and records that require correction.

2. Objectives

- To validate student identifiers and names.
- To check marks and attendance values against acceptable limits.
- To identify missing student or course information.
- To detect duplicate student IDs.
- To identify inconsistent academic records.
- To generate a report of valid and invalid records.
- To demonstrate the use of Python lists and tuples.

3. Assumptions

Field	Validation rule
Student ID	Must not be missing or duplicated
Student name	Must not be empty
Marks	Must be between 0 and 100
Attendance	Must be between 0% and 100%
Course	Must not be missing
Record format	Each record contains five fields

The sample records are for demonstration only. A record is considered valid only when all its fields pass the checks and its student ID is unique.

4. Algorithm

1. Start the program.
2. Store student records in a list of tuples.
3. Define the approved course names.
4. Check student IDs for duplicates.
5. Verify that names, marks, attendance, and courses are not missing.

6. Check whether marks and attendance are between 0 and 100.
7. Check whether each course belongs to the approved course list.
8. Add records without errors to the valid list.
9. Add records containing errors to the invalid list with their reasons.
10. Display valid records, invalid records, and a course-wise summary.
11. Stop the program.

5. Flowchart

Student Record Consistency Checker — Flowchart

Open or save the flowchart image

Save this image and insert it under Section 5 in your assignment PDF.

6. Pseudocode

START

STORE student records in a list of tuples

CREATE approved course list

CREATE valid records list

CREATE invalid records list

FOR each student record:

CREATE empty error list

CHECK whether student ID is duplicated

CHECK whether required fields are missing

CHECK whether marks are between 0 and 100

CHECK whether attendance is between 0 and 100

CHECK whether course is approved

IF error list is empty:

ADD record to valid records

ELSE:

ADD record and errors to invalid records

DISPLAY valid records

DISPLAY invalid records with reasons

DISPLAY course-wise summary

DISPLAY total valid and invalid records

STOP

7. Python Code

Copy the complete program into VS Code or Python IDLE and save it as student_record_checker.py.

```
# Integrated Student Record Consistency Checker

students = [
    ("STU1001", "Anu", 85, 90, "AIML"),
    ("STU1002", "Ravi", 110, 80, "CSE"),
    ("STU1003", "", 70, 75, "ECE"),
    ("STU1001", "Meena", 88, 95, "AIML"),
    ("STU1004", "Sita", 76, 105, "CSE"),
```

```
    ("STU1005", "Kiran", None, 80, ""),
    ("STU1006", "Priya", 92, 98, "MECH"),
    ("STU1007", "Arjun", 81, 88, "CSE")
]

approved_courses = ["AIML", "CSE", "ECE"]

valid_records = []
invalid_records = []

# Check each student record
for student in students:
    student_id, name, marks, attendance, course = student
    errors = []

    # Check duplicate student IDs
    if student_id is None or str(student_id).strip() == "":
        errors.append("Missing student ID")
    elif sum(1 for s in students if s[0] == student_id) > 1:
        errors.append("Duplicate student ID")

    # Check missing name
    if name is None or str(name).strip() == "":
        errors.append("Missing name")

    # Check marks
```

```
    if marks is None:
        errors.append("Missing marks")
    elif not isinstance(marks, (int, float)) or not 0 <= marks <= 100:
        errors.append("Invalid marks")

    # Check attendance
    if attendance is None:
        errors.append("Missing attendance")
    elif (not isinstance(attendance, (int, float))
          or not 0 <= attendance <= 100):
        errors.append("Invalid attendance")

    # Check course
    if course is None or str(course).strip() == "":
        errors.append("Missing course")
    elif course not in approved_courses:
        errors.append("Unapproved course")

    if len(errors) == 0:
        valid_records.append(student)
    else:
        invalid_records.append((student_id, errors))

# Display valid records
print("STUDENT RECORD CONSISTENCY REPORT")
print("--------------------------------")
```

```
print("\nVALID STUDENT RECORDS:")
for student in valid_records:
    print(student)

# Display invalid records
print("\nINVALID STUDENT RECORDS:")
for student_id, errors in invalid_records:
    print(student_id, ":", ", ".join(errors))

# Course-wise summary of valid records
print("\nCOURSE-WISE SUMMARY:")
for course in approved_courses:
    course_students = [
        s for s in valid_records if s[4] == course
    ]

    print(course, "- Students:", len(course_students))

    if len(course_students) > 0:
        average = sum(s[2] for s in course_students) / len(course_students)
        print("Average Marks:", round(average, 2))

print("\nTotal Records:", len(students))
print("Valid Records:", len(valid_records))
print("Invalid Records:", len(invalid_records))
```

8. Test Cases

Test case	Condition	Expected result
1	Valid ID, name, marks, attendance, and course	Valid record
2	Same student ID appears twice	Duplicate ID detected
3	Marks are 110	Invalid marks
4	Student name is empty	Missing name detected
5	Attendance is 105%	Invalid attendance
6	Marks are missing	Missing marks detected
7	Course is MECH	Unapproved course detected
8	All fields are valid and course is CSE	Valid record

9. Output Screenshot:

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:5
AMD64)] on win32
Enter "help" below or click "Help" above for more infc

= RESTART: C:/Users/Sushma Sri/AppData/Local/Programs/
y
STUDENT RECORD CONSISTENCY REPORT
---------------------------------

VALID STUDENT RECORDS:
('STU1007', 'Arjun', 81, 88, 'CSE')

INVALID STUDENT RECORDS:
STU1001 : Duplicate student ID
STU1002 : Invalid marks
STU1003 : Missing name
STU1001 : Duplicate student ID
STU1004 : Invalid attendance
STU1005 : Missing marks, Missing course
STU1006 : Unapproved course

COURSE-WISE SUMMARY:
AIML - Students: 0
CSE - Students: 1
Average Marks: 81.0
ECE - Students: 0

Total Records: 8
Valid Records: 1
Invalid Records: 7
```

10. Conclusion

The Integrated Student Record Consistency Checker validates student identifiers, names, marks, attendance, and course records. It detects duplicate IDs, missing information, invalid academic values, and unapproved courses. It also generates a course-wise summary of valid student records. This program demonstrates how Python lists, tuples, loops, conditional statements, and list operations can be used to maintain accurate academic records.

Case study7:

1. Problem Statement

A university conducts examinations in multiple subjects. Sometimes, the university applies approved moderation rules to adjust students' marks according to specified conditions. For example, a student who scores close to the passing mark may receive a small moderation adjustment.

The university needs a Python program to store original marks, apply moderation rules, ensure that marks remain within the permitted maximum, and compare original marks with moderated marks. The program should also display the total marks and result status before and after moderation

2. Objectives

- **To store students' marks across multiple subjects.**
- **To calculate original total marks.**
- **To apply specified moderation rules.**
- **To ensure moderated marks do not exceed the maximum marks.**
- **To compare original and moderated marks.**
- **To identify changes in pass or fail status.**
- **To generate a clear result moderation report.**

3. Assumptions

The following moderation rules are assumed for this demonstration only.

Parameter	Rule
Maximum marks per subject	100
Minimum passing marks	40
Maximum moderation added	5 marks per subject
Moderation eligibility	Original marks between 35 and 39
Maximum permitted marks after moderation	100

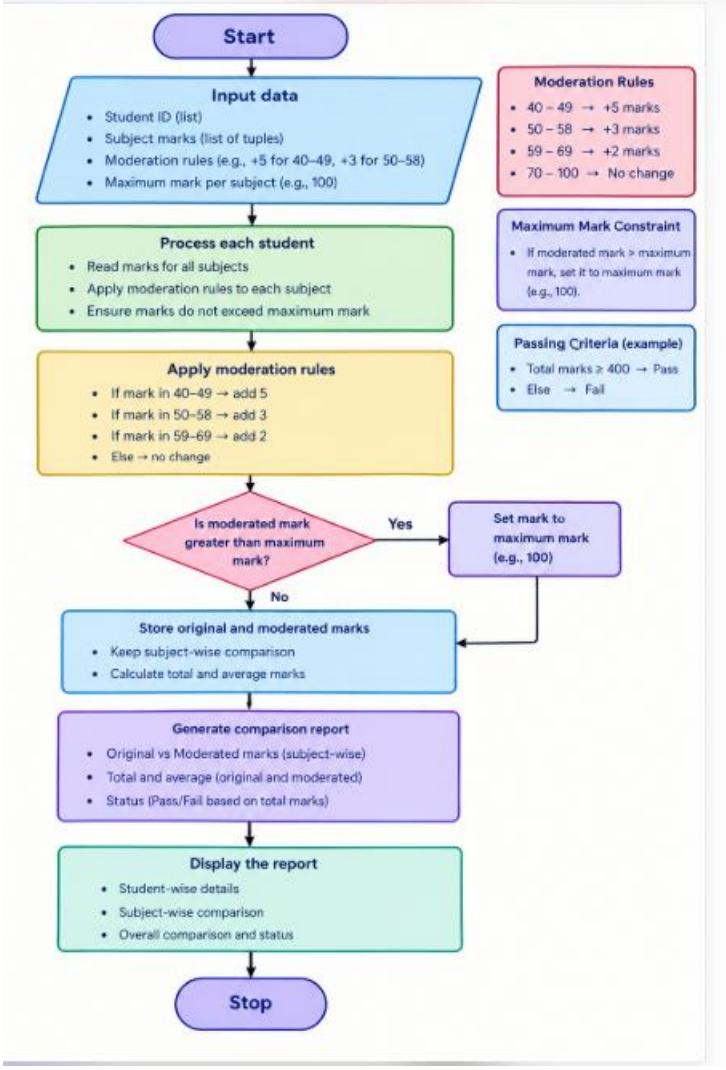
Moderation is applied only to students scoring 35–39 marks in a subject. Eligible students receive enough additional marks to reach 40, provided the adjustment does not exceed the five-mark moderation limit. These are sample rules, not an official university policy.

4. Algorithm

1. **Start the program.**
2. **Store student names and marks for multiple subjects.**
3. **Set the maximum marks to 100 and the passing marks to 40.**
4. **Calculate the original total and result status for each student.**
5. **Check each subject's marks against the moderation rules.**
6. **Add the permitted moderation marks to eligible subjects.**
7. **Ensure that no moderated mark exceeds 100.**
8. **Calculate the moderated total and result status.**
9. **Compare the original and moderated marks.**
10. **Display subject-wise marks, totals, and result changes.**
11. **Stop the program.**

5. Flowchart

University Examination Result Moderation — Flowchart



6. Pseudocode

START

SET maximum_marks = 100

SET passing_marks = 40

FOR each student:

STORE original marks

CALCULATE original total

CHECK original result

FOR each subject:

IF marks are between 35 and 39:

ADD enough marks to reach 40

IF moderated marks > 100:

SET marks = 100

CALCULATE moderated total

CHECK moderated result

DISPLAY original and moderated marks

DISPLAY totals and result status

STOP

7. Python Code

Copy the complete code into VS Code or Python IDLE and save it as result_moderation.py.

```
# University Examination Result Moderation

students = [
    ("Anu", (78, 36, 65, 88)),
    ("Ravi", (32, 45, 59, 70)),
```

```
    ("Sita", (39, 39, 92, 100)),
    ("Kiran", (85, 75, 68, 90))
]

maximum_marks = 100
passing_marks = 40

for student in students:
    name, original_marks = student
    moderated_marks = []

    # Apply moderation to each subject
    for mark in original_marks:
        if 35 <= mark < passing_marks:
            mark = mark + (passing_marks - mark)

        if mark > maximum_marks:
            mark = maximum_marks

        moderated_marks.append(mark)

    original_total = sum(original_marks)
    moderated_total = sum(moderated_marks)

    # A student passes only if every subject has 40 or more
    if all(mark >= passing_marks for mark in original_marks):
```

```
    original_result = "PASS"
  else:
    original_result = "FAIL"

  if all(mark >= passing_marks for mark in moderated_marks):
    moderated_result = "PASS"
  else:
    moderated_result = "FAIL"

  # Display comparison report
  print("\nStudent:", name)
  print("Original Marks:", original_marks)
  print("Moderated Marks:", moderated_marks)
  print("Original Total:", original_total)
  print("Moderated Total:", moderated_total)
  print("Original Result:", original_result)
  print("Moderated Result:", moderated_result)

  if original_result != moderated_result:
    print("Result Changed: Yes")
  else:
    print("Result Changed: No")
```

8. Test Cases

Student	Original marks	Expected moderated marks	Expected result

Anu	**78, 36, 65, 88**	**78, 40, 65, 88**	**Fail → Pass**
Ravi	**32, 45, 59, 70**	**32, 45, 59, 70**	**Fail → Fail**
Sita	**39, 39, 92, 100**	**40, 40, 92, 100**	**Fail → Pass**
Kiran	**85, 75, 68, 90**	**85, 75, 68, 90**	**Pass → Pass**

The test cases verify moderation eligibility, unchanged marks, result changes, and the maximum-mark constraint.

9. Output Screenshot

The following is the expected output. Run the program in VS Code or Python IDLE to capture your actual output screenshot for the PDF.

```
Enter "help" below or click "Help" above for more information.

== RESTART: C:/Users/Sushma Sri/AppData/Local/Programs/Python/Pyth

Student: Anu
Original Marks: (78, 36, 65, 88)
Moderated Marks: [78, 40, 65, 88]
Original Total: 267
Moderated Total: 271
Original Result: FAIL
Moderated Result: PASS
Result Changed: Yes

Student: Ravi
Original Marks: (32, 45, 59, 70)
Moderated Marks: [32, 45, 59, 70]
Original Total: 206
Moderated Total: 206
Original Result: FAIL
Moderated Result: FAIL
Result Changed: No

Student: Sita
Original Marks: (39, 39, 92, 100)
Moderated Marks: [40, 40, 92, 100]
Original Total: 270
Moderated Total: 272
Original Result: FAIL
Moderated Result: PASS
Result Changed: Yes

Student: Kiran
Original Marks: (85, 75, 68, 90)
Moderated Marks: [85, 75, 68, 90]
Original Total: 318
Moderated Total: 318
Original Result: PASS
Moderated Result: PASS
Result Changed: No
```

10. Conclusion

The University Examination Result Moderation program processes marks across multiple subjects and applies the specified moderation rule to eligible marks. It preserves the maximum-mark constraint and compares original and moderated results. The program also identifies students whose result status changes after moderation. It demonstrates the use of Python tuples, lists, loops, conditional statements, the sum() function, and the all() function.

Case study 8:

1. Problem Statement

Develop a Python program to calculate electricity bills for multiple apartments. The program should apply progressive electricity rates, add fixed charges, provide rebates to eligible apartments, detect discrepancies between calculated and recorded bills, and rank apartments from highest to lowest electricity consumption.

2. Objectives

- **Calculate electricity charges using progressive tariff slabs.**
- **Add fixed charges to each bill.**
- **Apply rebates to eligible apartments.**
- **Identify incorrect recorded bills.**
- **Rank apartments based on electricity consumption.**
- **Display a clear billing report.**

3. Assumptions

The following sample rules are used for this case study:

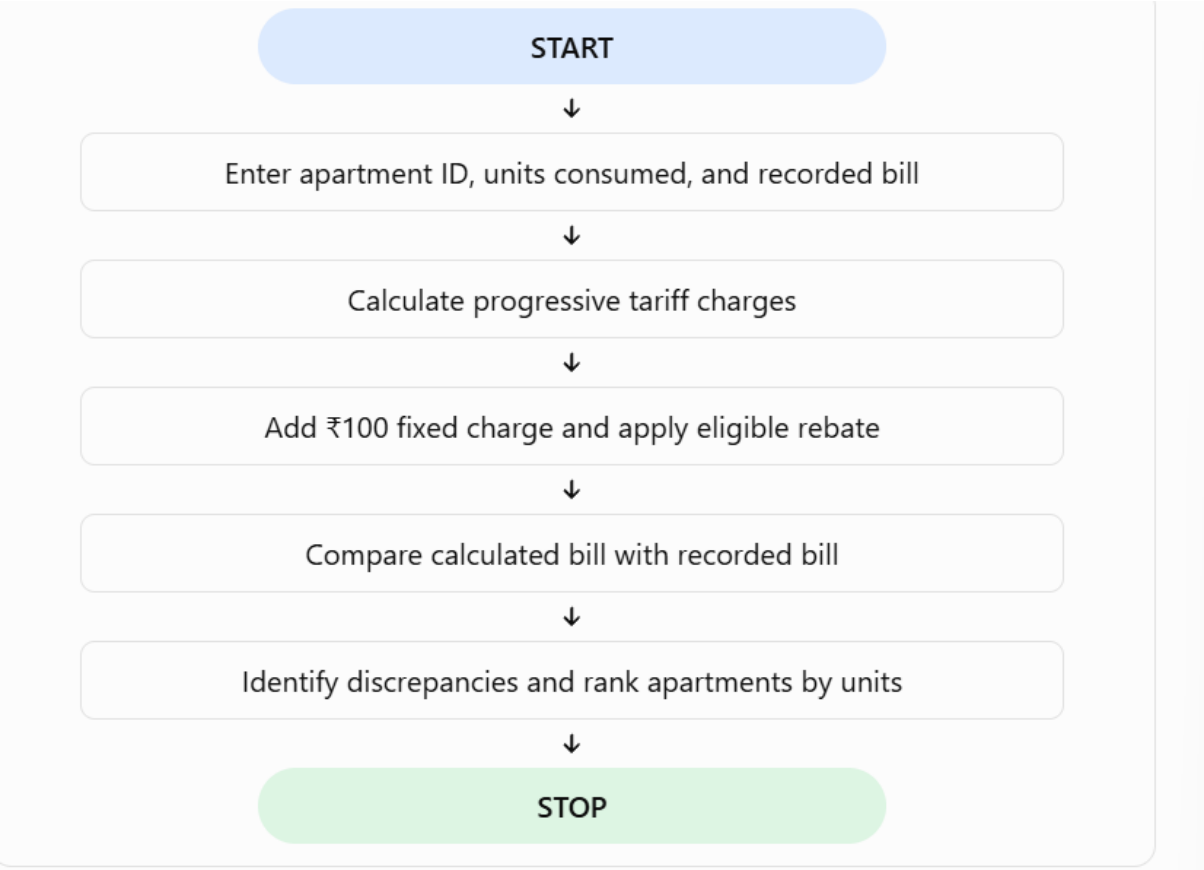
Electricity consumption	Rate
First 100 units	₹2 per unit
Next 100 units (101–200)	₹3 per unit
Above 200 units	₹5 per unit

- **Fixed charge: ₹100 per apartment.**
- **Rebate: ₹50 for apartments using 100 units or less.**
- **The recorded bill is compared with the calculated bill.**
- **An apartment is ranked by its total units consumed, from highest to lowest.**
- **All values are sample data for the assignment.**

4. Algorithm

1. **Start the program.**
2. **Store each apartment's ID, units consumed, and recorded bill.**
3. **Calculate the electricity charge using progressive tariff slabs.**
4. **Add the fixed charge of ₹100.**
5. **Subtract a rebate of ₹50 if consumption is 100 units or less.**
6. **Compare the calculated bill with the recorded bill.**
7. **Display any billing discrepancy.**
8. **Sort apartments by consumption in descending order.**
9. **Display the ranked list and billing summary.**
10. **Stop the program.**

5. Flowchart



6. Pseudocode

START

Store apartment details

FOR each apartment:

Read units and recorded bill

IF units <= 100:

charge = units * 2

ELSE IF units <= 200:

charge = 200 + (units - 100) * 3

ELSE:

charge = 500 + (units - 200) * 5

Add fixed charge of 100

IF units <= 100:

Subtract rebate of 50

Compare calculated bill with recorded bill

Display discrepancy if bills are different

Sort apartments by units in descending order

Display consumption ranking

STOP

7. Python Code

Copy this complete code into VS Code or IDLE

```
apartments = [
   ("A101", 80, 210),
   ("A102", 150, 470),
   ("A103", 250, 850),
   ("A104", 100, 250),
   ("A105", 220, 680)
]

fixed_charge = 100
rebate = 50

print("SMART ELECTRICITY BILLING SYSTEM")
print("--------------------------------")

for apartment in apartments:
   apartment_id, units, recorded_bill = apartment

   if units <= 100:
      charge = units * 2
   elif units <= 200:
      charge = 200 + (units - 100) * 3
   else:
      charge = 500 + (units - 200) * 5
```

```
    calculated_bill = charge + fixed_charge

    if units <= 100:
        calculated_bill = calculated_bill - rebate

    print("\nApartment:", apartment_id)
    print("Units consumed:", units)
    print("Calculated bill: Rs.", calculated_bill)
    print("Recorded bill: Rs.", recorded_bill)

    if calculated_bill == recorded_bill:
        print("Billing status: Correct")
    else:
        difference = recorded_bill - calculated_bill
        print("Billing status: Discrepancy")
        print("Difference: Rs.", difference)

print("\nCONSUMPTION RANKING")
print("--------------------")

ranked = sorted(apartments, key=lambda x: x[1],
                reverse=True)

rank = 1
for apartment in ranked:
    print(rank, apartment[0],
```

```
        "-", apartment[1], "units")
    rank = rank + 1
```

8. Test Cases

Apartment	Units	Recorded bill	Expected result
A101	80	₹210	Correct
A102	150	₹470	₹20 excess recorded
A103	250	₹850	Correct
A104	100	₹250	Correct
A105	220	₹680	₹20 less recorded

Additional test cases to check the tariff boundaries:

Units	Energy charge	Final bill
50	₹100	₹150 after rebate
100	₹200	₹250 after rebate
101	₹203	₹303
200	₹500	₹600
201	₹505	₹605

9. Sample Output

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:
AMD64)] on win32
Enter "help" below or click "Help" above for more inf

= RESTART: C:/Users/Sushma Sri/AppData/Local/Program
SMART ELECTRICITY BILLING SYSTEM
--------------------------------

Apartment: A101
Units consumed: 80
Calculated bill: Rs. 210
Recorded bill: Rs. 210
Billing status: Correct

Apartment: A102
Units consumed: 150
Calculated bill: Rs. 450
Recorded bill: Rs. 470
Billing status: Discrepancy
Difference: Rs. 20

Apartment: A103
Units consumed: 250
Calculated bill: Rs. 850
Recorded bill: Rs. 850
Billing status: Correct

Apartment: A104
Units consumed: 100
Calculated bill: Rs. 250
Recorded bill: Rs. 250
Billing status: Correct

Apartment: A105
Units consumed: 220
Calculated bill: Rs. 700
Recorded bill: Rs. 680
Billing status: Discrepancy
Difference: Rs. -20



CONSUMPTION RANKING
-------------------
1 A103 - 250 units
2 A105 - 220 units
3 A102 - 150 units
4 A104 - 100 units
5 A101 - 80 units
```

10. Conclusion

This Python program calculates electricity bills using progressive tariff rates, fixed charges, and rebates. It compares calculated bills with recorded bills to identify

discrepancies and ranks apartments according to their electricity consumption. The program demonstrates the practical use of conditional statements, lists, loops, sorting, and arithmetic operations in Python.

Case based9:

1. Problem Statement

Develop a Python program that processes hospital resource information stored in nested lists. The program should calculate available beds, identify shortages of staff and equipment, classify ward capacity, and prioritize cases based on their urgency and resource requirements.

2. Objectives

- **Store ward information using nested lists.**
- **Calculate available beds in each ward.**
- **Identify shortages of beds, staff, and equipment.**
- **Prioritize cases according to supplied rules.**
- **Display a ward-wise resource monitoring report.**

3. Assumptions

For this assignment, the hospital has three wards: General, Emergency, and ICU.

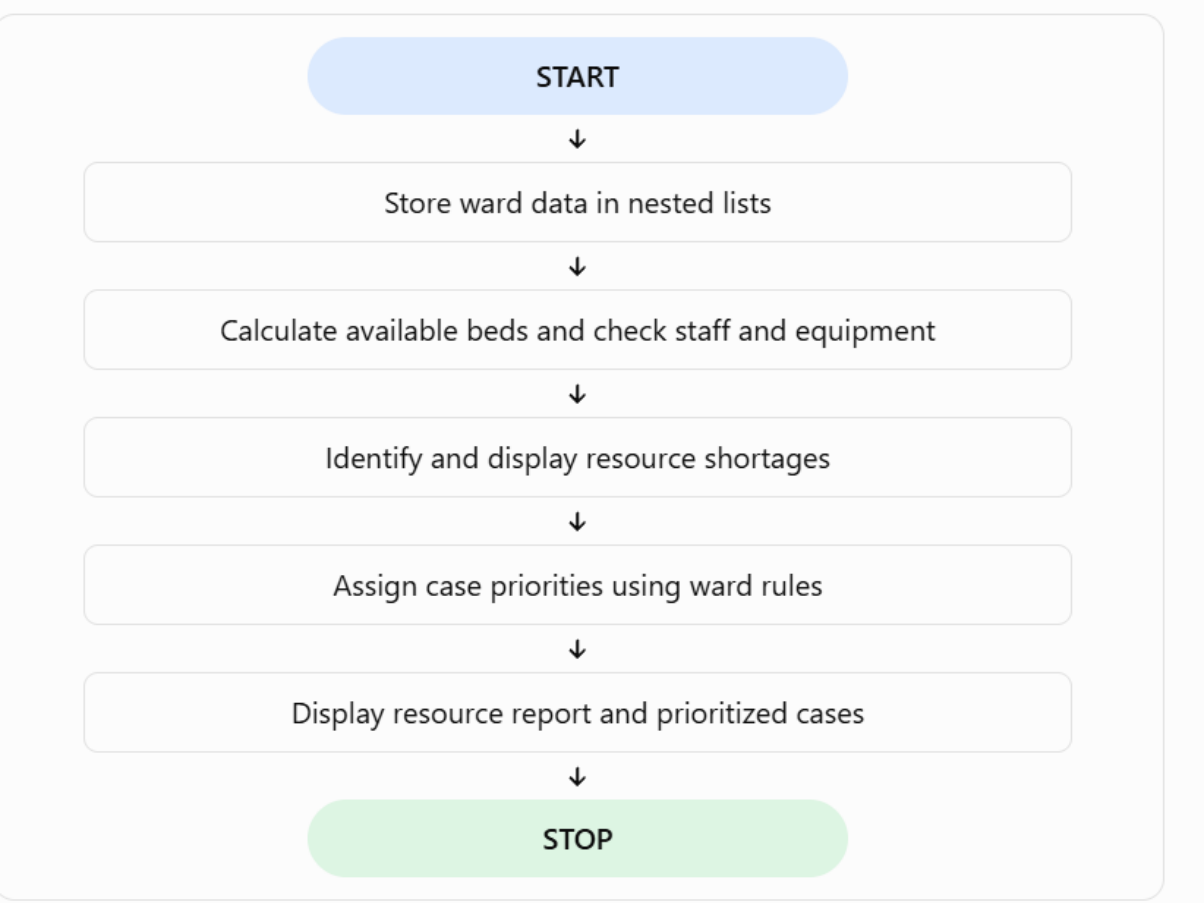
- **Each ward has a fixed bed capacity, current occupancy, available staff, and required staff.**
- **Each ward also has available equipment units and required equipment units.**
- **A shortage is reported whenever available resources are below the required amount.**
- **Cases are prioritized using the following rules:**
    - **Emergency cases receive High priority.**
    - **ICU cases receive High priority.**
    - **General ward cases receive Medium priority.**

- **These are simplified sample rules for a programming exercise, not real clinical triage guidance.**

4. Algorithm

1. **Start the program.**
2. **Store ward details in a nested list.**
3. **Read each ward's bed capacity, occupied beds, available staff, required staff, and equipment quantities.**
4. **Calculate available beds.**
5. **Check whether beds, staff, or equipment are insufficient.**
6. **Display the shortage details for each ward.**
7. **Store patient cases and their assigned wards.**
8. **Assign priority according to the predefined rules.**
9. **Display the prioritized case list.**
10. **Stop the program.**

5. Flowchart



6. Pseudocode

START

Store ward details in a nested list

FOR each ward:

Calculate available beds

IF occupied beds > bed capacity:

Display bed shortage

IF available staff < required staff:

Display staff shortage

IF available equipment < required equipment:

Display equipment shortage

Store patient cases and their wards

FOR each case:

IF ward is Emergency or ICU:

Priority = High

ELSE:

Priority = Medium

Display all prioritized cases

STOP

7. Python Code

Copy the complete program below into VS Code or IDLE.

```
# Hospital Resource Allocation System

wards = [
  ["General", 40, 35, 6, 8, 10, 12],
  ["Emergency", 20, 22, 8, 10, 7, 9],
  ["ICU", 10, 8, 5, 6, 4, 6]
```

```
]

print("HOSPITAL RESOURCE MONITORING")
print("----------------------------")

for ward in wards:
    name = ward[0]
    capacity = ward[1]
    occupied = ward[2]
    staff_available = ward[3]
    staff_required = ward[4]
    equipment_available = ward[5]
    equipment_required = ward[6]

    available_beds = capacity - occupied

    print("\nWard:", name)
    print("Bed capacity:", capacity)
    print("Occupied beds:", occupied)

    if available_beds < 0:
        print("Bed shortage:", abs(available_beds))
        available_beds = 0
    else:
        print("Available beds:", available_beds)
```

```
    if staff_available < staff_required:
        print("Staff shortage:",
              staff_required - staff_available)
    else:
        print("Staff status: Sufficient")

    if equipment_available < equipment_required:
        print("Equipment shortage:",
              equipment_required - equipment_available)
    else:
        print("Equipment status: Sufficient")

cases = [
    ["C101", "General"],
    ["C102", "Emergency"],
    ["C103", "ICU"],
    ["C104", "General"],
    ["C105", "Emergency"]
]

print("\nCASE PRIORITY REPORT")
print("--------------------")

for case in cases:
    case_id = case[0]
    ward_name = case[1]
```

```
    if ward_name == "Emergency" or ward_name == "ICU":
        priority = "High"
    else:
        priority = "Medium"

    print(case_id, "-", ward_name,
          "- Priority:", priority)
```

8. Test Cases

Test	Input condition	Expected result
1	General ward: 40 beds, 35 occupied	5 beds available
2	Emergency ward: 20 beds, 22 occupied	Shortage of 2 beds
3	General ward: 6 staff available, 8 required	Shortage of 2 staff
4	ICU: 4 equipment units available, 6 required	Shortage of 2 equipment units
5	Case C102 assigned to Emergency	High priority
6	Case C101 assigned to General	Medium priority

9. Sample Output

```
HOSPITAL RESOURCE MONITORING
----------------------------

Ward: General
Bed capacity: 40
Occupied beds: 35
Available beds: 5
Staff shortage: 2
```

Equipment shortage: 2

Ward: Emergency

Bed capacity: 20

Occupied beds: 22

Bed shortage: 2

Staff shortage: 2

Equipment shortage: 2

Ward: ICU

Bed capacity: 10

Occupied beds: 8

Available beds: 2

Staff shortage: 1

Equipment shortage: 2

CASE PRIORITY REPORT

C101 - General - Priority: Medium

C102 - Emergency - Priority: High

C103 - ICU - Priority: High

C104 - General - Priority: Medium

C105 - Emergency - Priority: High

10. Conclusion

This program uses Python nested lists to monitor hospital resources across different wards. It calculates available beds, identifies staff and equipment

shortages, and assigns case priorities using predefined rules. It demonstrates the practical use of lists, loops, conditional statements, and arithmetic operations in hospital resource management

Case based10:

1. Problem Statement

Develop a Python program that processes product demand, available stock levels, order quantities, and priority categories. The program should prioritize orders, determine the quantity that can be fulfilled from available stock, identify whether each order can be fully fulfilled, and calculate any outstanding quantity.

2. Objectives

- **Store product stock and order details using Python lists.**
- **Prioritize orders based on their priority categories.**
- **Calculate fulfilled quantities for each order.**
- **Identify fully fulfilled and partially fulfilled orders.**
- **Calculate outstanding quantities.**
- **Display a clear warehouse order report.**

3. Assumptions

The following sample rules are used:

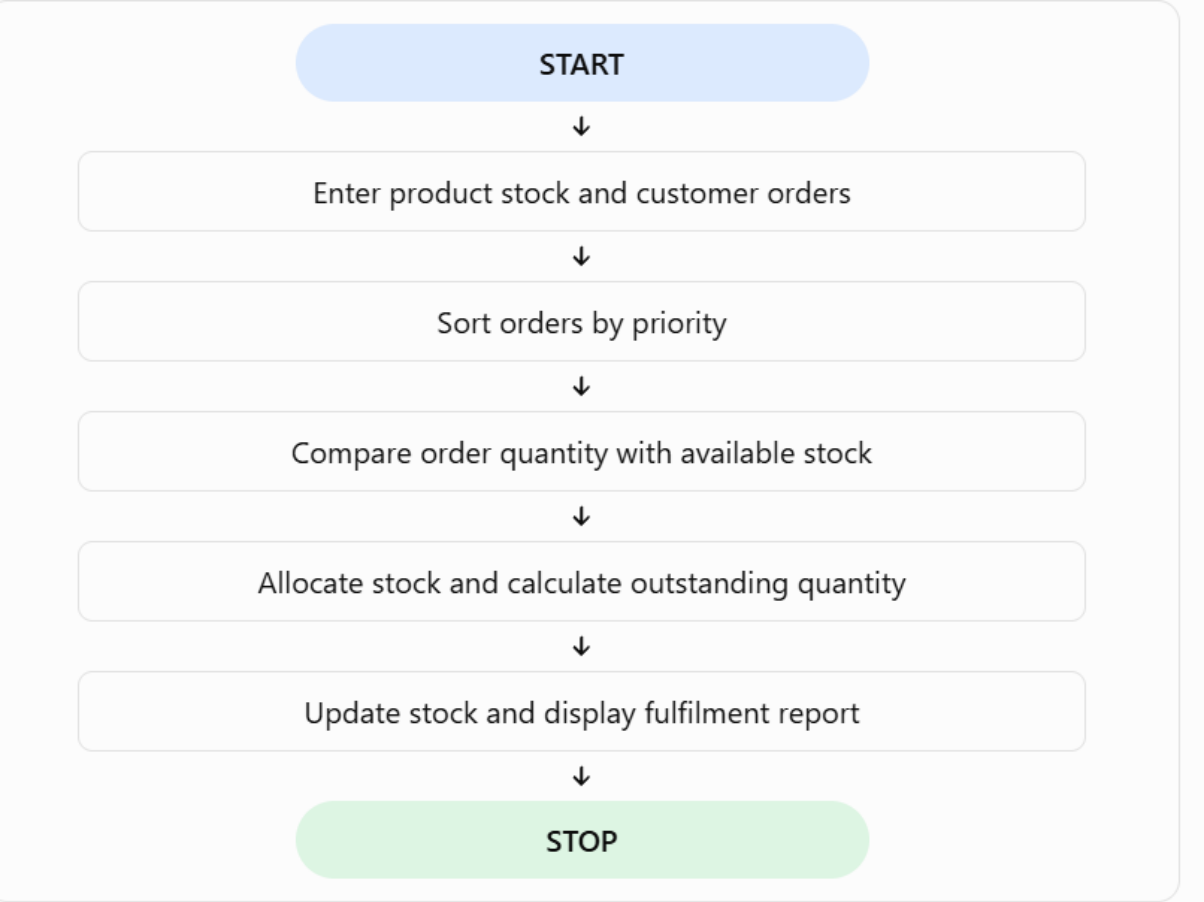
- **High priority orders are processed first.**
- **Medium priority orders are processed second.**
- **Low priority orders are processed last.**
- **Orders with the same priority are processed in their original order.**
- **Available stock is allocated to orders until it runs out.**
- **Outstanding quantity = Order quantity – Fulfilled quantity.**
- **Product stock cannot become negative.**

4. Algorithm

1. **Start the program.**

2. **Store product stock levels.**
3. **Store customer orders with order ID, product, quantity, and priority.**
4. **Sort orders by priority: High, Medium, then Low.**
5. **Check the available stock for each product.**
6. **Fulfil as much of each order as the available stock allows.**
7. **Update the remaining stock.**
8. **Calculate the outstanding quantity for each order.**
9. **Display the fulfilment status and remaining stock.**
10. **Stop the program.**

5. Flowchart



6. Pseudocode

START

```
START

Store product stock levels
Store customer orders

Sort orders by priority:
  High, Medium, Low

FOR each order:
  Read product, quantity, and priority
  Check available stock

  IF stock >= order quantity:
    fulfilled = order quantity
  ELSE:
    fulfilled = available stock

  outstanding = order quantity - fulfilled
  Update remaining stock

  IF outstanding == 0:
    Display Fully Fulfilled
  ELSE IF fulfilled > 0:
    Display Partially Fulfilled
  ELSE:
    Display Not Fulfilled
```

Display all order details

Display remaining stock

STOP

7. Python Code

```
# Integrated Warehouse Order Prioritization

stock = {
   "Laptop": 5,
   "Mouse": 10,
   "Keyboard": 4
}

orders = [
   ["O101", "Laptop", 4, "High"],
   ["O102", "Mouse", 7, "Medium"],
   ["O103", "Laptop", 3, "Low"],
   ["O104", "Keyboard", 2, "High"],
   ["O105", "Mouse", 5, "High"],
   ["O106", "Keyboard", 3, "Medium"]
]

priority_order = {
```

```
    "High": 1,
    "Medium": 2,
    "Low": 3
}

orders.sort(key=lambda order: priority_order[order[3]])

print("WAREHOUSE ORDER REPORT")
print("----------------------")

for order in orders:
    order_id = order[0]
    product = order[1]
    quantity = order[2]
    priority = order[3]

    available = stock[product]

    if available >= quantity:
        fulfilled = quantity
    else:
        fulfilled = available

    outstanding = quantity - fulfilled
    stock[product] = available - fulfilled
```

```
    if outstanding == 0:
        status = "Fully Fulfilled"
    elif fulfilled > 0:
        status = "Partially Fulfilled"
    else:
        status = "Not Fulfilled"

    print("\nOrder ID:", order_id)
    print("Product:", product)
    print("Priority:", priority)
    print("Ordered quantity:", quantity)
    print("Fulfilled quantity:", fulfilled)
    print("Outstanding quantity:", outstanding)
    print("Status:", status)

print("\nREMAINING STOCK")
print("---------------")

for product in stock:
    print(product, ":", stock[product], "units")
```

8. Test Cases

Order ID	Product	Ordered	Priority	Expected result
O101	Laptop	4	High	Fully fulfilled
O104	Keyboard	2	High	Fully fulfilled

O105	Mouse	5	High	Fully fulfilled
O102	Mouse	7	Medium	5 fulfilled, 2 outstanding
O106	Keyboard	3	Medium	2 fulfilled, 1 outstanding
O103	Laptop	3	Low	1 fulfilled, 2 outstanding

9. Sample Output

WAREHOUSE ORDER REPORT

Order ID: O101

Product: Laptop

Priority: High

Ordered quantity: 4

Fulfilled quantity: 4

Outstanding quantity: 0

Status: Fully Fulfilled

Order ID: O104

Product: Keyboard

Priority: High

Ordered quantity: 2

Fulfilled quantity: 2

Outstanding quantity: 0

Status: Fully Fulfilled

Order ID: O105

Product: Mouse

Priority: High

Ordered quantity: 5

Fulfilled quantity: 5

Outstanding quantity: 0

Status: Fully Fulfilled

Order ID: O102

Product: Mouse

Priority: Medium

Ordered quantity: 7

Fulfilled quantity: 5

Outstanding quantity: 2

Status: Partially Fulfilled

Order ID: O106

Product: Keyboard

Priority: Medium

Ordered quantity: 3

Fulfilled quantity: 2

Outstanding quantity: 1

Status: Partially Fulfilled

Order ID: O103

Product: Laptop

Priority: Low

Ordered quantity: 3

Fulfilled quantity: 1

Outstanding quantity: 2

Status: Partially Fulfilled

REMAINING STOCK

Laptop : 0 units

Mouse : 0 units

Keyboard : 0 units

10. Conclusion

This Python program manages warehouse orders according to their priority categories. It checks available stock, calculates fulfilled and outstanding quantities, updates inventory, and identifies fully or partially fulfilled orders. It demonstrates the use of dictionaries, lists, loops, conditional statements, and sorting in a practical warehouse management system.

Done by:

SUSHMA SRI POUJULAPATI

AIML 1